Sprint 2 AI Usage Log

Project: CareerConnect — Job Search and Application Tracking Platform
Student: Michael Theodore
Sprint 2: Fall 2026
AI Tool: ChatGPT (OpenAI)
Repository: https://github.com/Nurtrantpoem19/soen-341

Task 2.1: JWT Authentication Integration

Purpose of AI Use:
Implementation guidance and debugging for user authentication.

Chat Link or Prompt/Response:
Used ChatGPT to help implement and test registration, login, JWT generation, and authentication middleware in the existing Express backend.
"Help me implement real JWT authentication in my existing Node.js and Express backend. I need registration, login, password hashing, and middleware to protect user-specific API endpoints."

AI-Suggested Content:
ChatGPT suggested using bcrypt for password hashing, JWT tokens for login sessions, and middleware to protect API endpoints.

Validation:
Tested registration and login requests against the local PostgreSQL database. Verified that protected routes rejected requests without valid authentication.

Decision:
Accepted the authentication approach and integrated it into the existing backend.

Reflection:
AI helped explain how JWT authentication connects to the existing API. Testing was necessary to confirm that authentication worked with our database and routes.

Task 2.2: Resume Management API

Purpose of AI Use:
Backend implementation assistance and file-upload validation.

Chat Link or Prompt/Response:
Asked ChatGPT to guide the implementation of resume uploading, listing, downloading, and replacement, one step at a time.
“Guide me through implementing a resume management API using Express, Multer, and PostgreSQL. Users should be able to upload, list, download, and replace their resumes. Explain each implementation step.”

AI-Suggested Content:
ChatGPT provided implementation suggestions using Multer, PostgreSQL, and filesystem operations. The API supports PDF and DOCX uploads, a 5 MB size limit, and authenticated access to user-owned resumes.

Validation:
Manually tested uploading, downloading, and replacing a PDF. Compared file hashes to confirm that downloaded content matched the uploaded file. Also tested invalid files, unsupported formats, and oversized uploads.

Decision:
Accepted the implementation after testing the endpoints and correcting issues encountered during development.

Reflection:
AI was useful for breaking the feature into smaller implementation steps. Manual testing helped confirm that the API handled actual files correctly.

Task 2.3: Automated Backend Testing

Purpose of AI Use:
Generating test cases and verifying backend functionality.

Chat Link or Prompt/Response:
Asked ChatGPT to help extend `test/database.test.js` with automated resume-management tests.
“Help me add automated tests for the resume endpoints in `test/database.test.js`.”

AI-Suggested Content:
ChatGPT suggested tests for resume uploading, listing, downloading, replacement, authentication, ownership restrictions, invalid PDFs, file-size limits, and unsupported file types.

Validation:
Ran `node --check test/database.test.js` and `npm.cmd test` after adding tests. The final test execution reported one passing top-level test and zero failures.

Decision:
Accepted the test cases after verifying their results and making necessary corrections.

Reflection:
The automated tests helped verify that backend functionality continued working after changes. AI accelerated test development, but the actual results had to be checked locally.

Task 2.4: Debugging Resume File Cleanup

Purpose of AI Use:
Troubleshooting and correcting temporary file cleanup during automated testing.

Chat Link or Prompt/Response:
Asked ChatGPT to investigate why the number of files in `uploads/resumes` increased after each test run.
“Why are uploaded resume files still accumulating after running my automated tests?”

AI-Suggested Content:
ChatGPT initially suggested querying resume records during test cleanup. After further debugging, it suggested saving the uploaded file path before the database record disappeared and deleting the file directly during cleanup.

Validation:
Used PowerShell to count files before, during, and after testing. Inspected PostgreSQL records and added temporary debugging statements. Some initial AI suggestions introduced syntax errors or did not resolve the issue, so they were revised.

The final verification showed 20 files during testing and 19 afterward, confirming that the newly created test file was removed.

Decision:
Modified the initial suggestions and accepted the final cleanup approach after successful testing.

Reflection:
This task showed why AI-generated fixes should not be accepted without verification. The first suggestions did not fully solve the problem, and examining the actual database records and test output was necessary to identify the cause.

Task 2.5: Backend API Documentation

Purpose of AI Use:
Documentation drafting and frontend integration support.

Chat Link or Prompt/Response:
Asked ChatGPT to prepare backend API documentation using the actual endpoints and validation rules in `server.js`.

AI-Suggested Content:
ChatGPT helped prepare `docs/sprint_2/backend_api.md`, covering authentication, profile management, resume management, request examples, and expected HTTP responses.
"Help me document my backend API using the actual routes and validation rules in `server.js`."

Validation:
Checked the documentation against the route definitions and validation logic in `server.js`, including required profile fields, JWT authentication, supported resume formats, and file-size limits.

Decision:
Accepted the documentation and added it to the Sprint 2 project files.

Reflection:
AI helped organize technical information into a format that frontend teammates could use. Reviewing the actual backend code was important to avoid documenting endpoints that had not been implemented.

Task 2.6: GitHub Integration and Pull Request

Purpose of AI Use:
Git workflow guidance and repository integration.

Chat Link or Prompt/Response:
Asked ChatGPT how to commit backend changes, upload files manually through GitHub, and merge the feature branch into `main`.

AI-Suggested Content:
ChatGPT suggested reviewing Git status, staging only the intended files, committing changes on `feature/sprint2-backend`, and using a Pull Request instead of directly overwriting `main`.

Validation:
Reviewed the six changed files, created local commit `8288ab7`, uploaded changes through GitHub, and checked that the Pull Request had no merge conflicts. GitHub subsequently confirmed that the Pull Request was successfully merged and closed.

Decision:
Accepted the Pull Request workflow after confirming the merge was allowed and coordinating with teammates.

Reflection:
AI helped clarify the Git workflow and reduce the risk of accidentally modifying teammates' work. The final merge was verified directly on GitHub.

Overall Reflection

During Sprint 2, I used ChatGPT mainly for backend implementation guidance, debugging, automated testing, documentation, and GitHub workflow assistance.

The most useful aspect was being able to work through technical problems one step at a time and verify each change before continuing.

Not every AI suggestion worked immediately. The resume cleanup issue required several debugging steps, and some generated code needed corrections. I relied on terminal output, database queries, automated tests, and manual API testing to validate the final implementation.

AI was used as a development assistant rather than as a replacement for testing or reviewing the code.